PepPro: purchase tracking is firing, but without an order value

Prepared 7 September 2026 for the PepPro development team. Business `biz_oYbMas6NGuxzJp` / `pepprogroup.com`.

Short version. There are two separate problems here, and the second one is the expensive one. Only **one** purchase event has been recorded in the last ten days, and that single event arrived **with no order value attached**. A purchase with no value cannot be used to optimize a campaign, so as far as ad reporting is concerned this store currently has no usable purchase data at all.

1. What was measured, and how

Every figure in this document was read live from the Whop events API on 7 September 2026, over a 28 August 2026 to 7 September 2026 (ten days) window. The sweep followed the API's paging cursors to the end and returned **1,659 events across 4 pages**. A partial read was not accepted: the tool used refuses to write a result at all if any page fails, because presence survives a truncated list but **absence does not**.

> **One reading trap worth naming, because it silently produces a wrong answer.** On `GET /events` every custom event, purchases included, comes back with `event_name: "pixel.custom"`. The real name lives in a separate `custom_name` field. Filtering on `event_name` for "purchase" returns zero even on an account that provably has purchases. Every count below was taken across both fields.

2. Where purchases stand today

what	measured
Purchase events in the window	1
That event's value	**null**
Page it fired from	`/orders/<uuid>`
Event id shape	`biz_oYbMas6NGuxzJp:<integer>`
Add to cart events	8
Test events still in the account	8, named `apicheck_ignore_me`

> **The value is the finding.** Whop accepted the event, so nothing looks broken from the outside, and the account shows a purchase. But the row carries `total_usd_amount: null`. Anything that reports on revenue reads that as a sale worth nothing, which is indistinguishable from a sale worth zero. Whatever assembles this payload is not reading the order total, or is reading it before the total exists.

The eight `apicheck_ignore_me` rows are leftover test events. They are harmless but they are sitting in the same account as real data, and two of them carry dollar values, so they are worth removing from whatever emits them.

3. The second problem: it fires from the browser

The one purchase that did arrive was raised by the buyer's browser. Established on your own data: the purchasing visitor also registered a page view at the moment of the sale, and the event carries a resolved city, both of which only a real browser request produces.

> **Why one purchase in ten days is itself suspicious.** This account recorded eight add to cart events over the same window. Carts do not usually outnumber completed orders by that margin without something in between failing to report. A browser fired purchase event is exactly the kind of thing that fails quietly: it reports nothing when the customer closes the tab on the payment screen, when a blocker stops the request, or when payment confirms asynchronously later. Moving the event to the server both fixes the value and removes that whole class of silent loss.

4. What to build

One server side call, made from wherever your code already knows an order is paid for. The contract is:

```
POST https://api.whop.com/api/v1/events
Authorization: Bearer <your Whop API key>
Content-Type: application/json

{
  "event_name": "whop_purchase",
  "company_id": "biz_oYbMas6NGuxzJp",
  "event_id":   "<stable unique id for this order>",
  "value":      129.99,
  "currency":   "usd",
  "url":        "https://www.pepprogroup.com/...",
  "user":    { "anonymous_id": "<_wuid cookie>", "email": "buyer@example.com" },
  "context": { "user_agent": "<the buyer's user agent>" }
}
```

Note that on the **send** side the event name goes in `event_name`. On the read side the same event comes back as `pixel.custom` with the name in `custom_name`. That asymmetry is Whop's, not a mistake in this document.

Read the total from the paid order, not from the cart or the page. The current event's null value is almost certainly a payload assembled before the order total was settled. Sending this from the server, at the point where payment is already confirmed, means the real total is available by definition.

4a. The attribution join, which is the part that is easy to miss

A server side event with no `user.anonymous_id` is a sale Whop cannot connect to the ad that produced it. It will be counted as revenue and attributed to nothing. The value comes from the `_wuid` cookie that the Whop pixel already sets in the buyer's browser, so it has to be captured in the browser and carried through to the server.

```
/* Runs in the browser at checkout, BEFORE the order is submitted. */
/* _wuid is the visitor id the Whop pixel already sets on your site. */
/* Without it the server event cannot be joined to the ad click, and */
/* the sale reports as unattributed. */
function whopWuid() {
  const m = document.cookie.match(/(?:^|;\s*)_wuid=([^;]+)/);
  return m ? decodeURIComponent(m[1]) : null;
}

/* Persist it ON THE ORDER RECORD, alongside the order id, so the */
/* server still has it when the payment confirms minutes later. */
orderPayload.whop_wuid = whopWuid();
```

Capture it at checkout, not at confirmation. If the payment confirms after the customer has closed the tab, there is no browser left to read the cookie from. The whole point of moving this server side is that the tab is no longer required, and reading the cookie late gives that dependency straight back.

4b. The endpoint

```
/* app/api/whop-purchase/route.js  Next.js App Router on Vercel. */
/* Call this once from the code path that CONFIRMS an order, server side. */
/* Never from the browser: the customer's tab is not involved. */

/* Server env var only. Never NEXT_PUBLIC_. */
const WHOP_KEY = process.env.WHOP_API_KEY;
const COMPANY  = 'biz_oYbMas6NGuxzJp';

export async function POST(req) {
  const order = await req.json();

  /* event_id is what makes this safe to call more than once. Whop */
  /* de-duplicates on it, so a retry, a refresh or a replayed webhook */
  /* cannot double count the same order. */
  const body = {
    event_name: 'whop_purchase',
    company_id: COMPANY,
    event_id:   'purchase_' + order.id,
    value:      Number(order.total),
    currency:   'usd',
    url:        'https://www.pepprogroup.com/orders/' + order.id,
    user: {
      anonymous_id: order.whop_wuid || undefined,
      email:        order.email     || undefined
    },
    context: { user_agent: order.user_agent || undefined }
  };

  const r = await fetch('https://api.whop.com/api/v1/events', {
    method:  'POST',
    headers: {
      'Authorization': 'Bearer ' + WHOP_KEY,
      'Content-Type':  'application/json'
    },
    body: JSON.stringify(body)
  });

  if (!r.ok) {
    /* Log it and retry out of band. Do NOT fail the customer's order */
    /* because a tracking call failed. */
    console.error('whop purchase post failed', r.status, await r.text());
  }
  return new Response(null, { status: 204 });
}
```

Keep the API key server side. On Vercel that means a plain environment variable, not one prefixed `NEXT_PUBLIC_`. A key exposed to the browser can be used by anyone to write events into your account.

5. How to confirm it works

Place one real order, then read the account back:

```
curl -s -H "Authorization: Bearer $WHOP_API_KEY" \\
  "https://api.whop.com/api/v1/events?company_id=biz_oYbMas6NGuxzJp&from=2026-09-
07T00:00:00Z&to=2026-09-08T00:00:00Z"
```

You are looking for a row with `custom_name: "whop_purchase"` carrying the order's real total in `total_usd_amount`. Two further checks separate a real server side install from a browser one:

1. **Send it twice.** Post the same `event_id` again. The event count must not go up. If it does, the idempotency key is not being set from anything stable.
2. **Kill the tab.** Complete a checkout and close the browser before the confirmation page renders. The purchase event must still arrive. This is the entire reason for the work, and it is the one test a browser install cannot pass.

> Whop's event stream lags ingestion by a few minutes, and an empty read taken too soon is not evidence of a failure. If a window looks empty, widen it to several days before concluding anything. A single empty response has produced more than one false alarm on this platform.